SR UNIVERSITY

AI Assistant Coding
Assignment-11.4

Name of Student : V. Abhinav Batch : 41
Enrollment No. : 2303A52486

Task 1: Stack Implementation for Undo Operations (LIFO)

- Use an AI coding assistant to help implement a Stack class in Python with the following methods:
  - push(action)
  - pop()
  - peek()
  - is_empty()
- Prompt the AI to:
  - Generate a clean class skeleton with docstrings
  - Explain why a stack is suitable for undo functionality
  - Suggest an alternative implementation using collections.deque

Prompt:
Implement a stack class with the following methods: push, pop, and is_empty with docstring comments.

Original Code:

Assign 11.4.py > Stack > push

```python
# Implement a stack class with the following methods: push, pop, and is_em
class Stack:
    def __init__(self):
        self.items = []
    def push(self, item):
        """
        Push an item onto the stack.
        """
        self.items.append(item)
    def pop(self):
        """
        Pop the top item from the stack.
        """
        return self.items.pop()
    def is_empty(self):
        """
        Check if the stack is empty.
        """
        return len(self.items) == 0
    def size(self):
        """
        Get the size of the stack.
        """
        return len(self.items)
    def peek(self):
        """
        Get the top item from the stack.
        """
        return self.items[-1]
```

```python
    def __str__(self):
        """
        Return a string representation of the stack.
        """
        return str(self.items)

# Create a stack and push 1, 2, 3, 4, 5 onto it.
stack = Stack()
stack.push(1)
stack.push(2)
stack.push(3)
stack.push(4)
stack.push(5)
print(stack)

# Pop the top item from the stack and print it.
print(stack.pop())
print(stack)
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:/Us
sers/Abhi/Documents/AI Assistant Coding/Assign 11.4.py"
[1, 2, 3, 4, 5]
5
[1, 2, 3, 4]
```

Alternative using collections.deque:

Assign 11.4.py > ...

```python
import collections
class Stack:
    def __init__(self):
        """
        Initialize a stack using collections.deque.
        """
        self.items = collections.deque()
    def push(self, item):
        """
        Push an item onto the stack.
        """
        self.items.append(item)
    def pop(self):
        """
        Pop the top item from the stack.
        """
        return self.items.pop()
    def is_empty(self):
        """
        Check if the stack is empty.
        """
        return len(self.items) == 0
    def size(self):
        """
        Get the size of the stack.
        """
        return len(self.items)
    def peek(self):
        """
        Get the top item from the stack.
        """
        return self.items[-1]
```

```python
    def __str__(self):
        """
        Return a string representation of the stack.
        """
        return str(self.items)

# Create a stack and push 1, 2, 3, 4, 5 onto it.
stack = Stack()
stack.push(1)
stack.push(2)
stack.push(3)
stack.push(4)
stack.push(5)
print(stack)

# Pop the top item from the stack and print it.
print(stack.pop())
print(stack)
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding
ts/AI Assistant Coding/Assign 11.4.py"
deque([1, 2, 3, 4, 5])
5
deque([1, 2, 3, 4])
```

Explanation:

This task implements a stack class to simulate undo functionality, where the most recent action is removed first. The output shows items being pushed and popped correctly, confirming proper LIFO behavior (pages 1–2). A list-based stack is simple and efficient for push/pop at the end, while a deque-based stack offers similar O(1) operations but is more memory-efficient and consistent for large workloads. AI correctly suggested deque as an alternative because it is optimized for fast insertions/removals. This demonstrates how AI can propose better structural choices beyond basic implementation.

Task 2: Queue for Customer Service Requests (FIFO)

Task Description:

- Implement a Queue class with:
  - enqueue(request)
  - dequeue()
  - is_empty()
- First, use a Python list-based approach.
- Then, ask AI to:
  - Review the performance implications of list-based queues
  - Suggest and implement an optimized version using
  - collections.deque
  - Explain why deque performs better for queues

Prompt:

Implement a queue class with the following methods: enqueue, dequeue, and is_empty with docstring comments.

Code:

```python
class Queue:
    def __init__(self):
        """
        Initialize a queue using collections.deque.
        """
        self.items = collections.deque()
    def enqueue(self, item):
        """
        Enqueue an item onto the queue.
        """
        self.items.append(item)
    def dequeue(self):
        """
        Dequeue an item from the queue.
        """
        return self.items.popleft()
    def is_empty(self):
        """
        Check if the queue is empty.
        """
        return len(self.items) == 0
    def size(self):
        """
        Get the size of the queue.
        """
        return len(self.items)
    def __str__(self):
        """
        Return a string representation of the queue.
        """
        return str(self.items)

# Create a queue and enqueue 1, 2, 3, 4, 5 onto it.
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
queue.enqueue(3)
```

```python
# Create a queue and enqueue 1, 2, 3, 4, 5 onto it.
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
queue.enqueue(3)

print(queue)

# Dequeue an item from the queue and print it.
print(queue.dequeue())
print(queue)

# Check if the queue is empty and print the result.
print(queue.is_empty())
```

Output:

```
ts/AI Assistant Coding/Assign 11.4.py"
deque([1, 2, 3, 4, 5])
5
deque([1, 2, 3, 4])
deque([1, 2, 3])
1
deque([2, 3])
False
```

Alternative using collections.deque:

```python
Assign 11.4.py > ...
# Suggest and implement an optimized version using collections.deque.
import collections
class Queue:
    def __init__(self):
        """
        Initialize a queue using collections.deque.
        """
        self.items = collections.deque()
    def enqueue(self, item):
        """
        Enqueue an item onto the queue.
        """
        self.items.append(item)
    def dequeue(self):
        """
        Dequeue an item from the queue.
        """
        return self.items.popleft()
    def is_empty(self):
        """
        Check if the queue is empty.
        """
        return len(self.items) == 0
    def size(self):
        """
        Get the size of the queue.
        """
        return len(self.items)
    def __str__(self):
        """
        Return a string representation of the queue.
        """
        return str(self.items)
```

```
(variable) queue: Queue
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
queue.enqueue(3)
queue.enqueue(4)
queue.enqueue(5)
print(queue)

# Dequeue an item from the queue and print it.
print(queue.dequeue())
print(queue)

# Check if the queue is empty and print the result.
print(queue.is_empty())

# Get the size of the queue and print it.
print(queue.size())

# Return a string representation of the queue.
print(queue.__str__())
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:/Users/Abhi/AppData/Local
ts/AI Assistant Coding/Assign 11.4.py"
deque([1, 2, 3, 4, 5])
1
deque([2, 3, 4, 5])
False
4
deque([2, 3, 4, 5])
```

Explanation:

This task builds a queue using both a list approach and a deque-based approach to process requests in FIFO order. The outputs show correct enqueue/dequeue behavior, proving functional correctness of both implementations (pages 5–6). AI-assisted performance comparison explains that list-based queues require O(n) time for removing the first element, while deque provides O(1) operations using optimized double-ended structure. In real systems with many users, deque scales better and avoids shifting overhead. This explains why deque is preferred for queues in production applications.

Task 3: Singly Linked List for Dynamic Playlist Management

Task Description:

- **Implement a Singly Linked List with:**
  - **insert_at_end(song)**
  - **delete_value(song)**
  - **traverse()**
- **Use AI to:**
  - **Add inline comments explaining pointer manipulation**
  - **Highlight tricky parts of insertion and deletion**
  - **Suggest edge case test scenarios (empty list, single node, deletion at head)**

Prompt:

Implement a linked list class with the following methods: insert, delete, and search, Add inline comments explaining pointer manipulation, Highlight tricky parts of insertion and deletion. Also Suggest edge case test scenarios (empty list, single node,deletion at head)

Code:

Assign 11.4.py > LinkedList > insert

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            current = self.head
            while current.next is not None:
                current = current.next
            current.next = new_node
    def delete(self, data):
        if self.head is None:
            return
        if self.head.data == data:
            self.head = self.head.next
            return
        current = self.head
        while current.next is not None:
            if current.next.data == data:
                current.next = current.next.next
                return
            current = current.next
```

Assign 11.4.py > ...

```python
class LinkedList:
    def __str__(self):
        while current is not None:
            result.append(current.data)
            current = current.next
        return result
    def print_list(self):
        current = self.head
        while current is not None:
            print(current.data, end=' ')
            current = current.next
        print()

# Test cases:
#  1. Empty list
linked_list = LinkedList()
linked_list.print_list()
# 2. Single node
linked_list = LinkedList()
linked_list.insert(1)
linked_list.print_list()
# 3. Deletion at head
linked_list.delete(1)
linked_list.print_list()
# 4. Deletion in middle
linked_list.insert(2)
linked_list.insert(3)
linked_list.delete(2)
linked_list.print_list()
# 5. Deletion at end
linked_list.insert(4)
linked_list.delete(4)
linked_list.print_list()
# 6. Deletion of non-existent node
linked_list.delete(5)
linked_list.print_list()
# 7. Deletion of multiple nodes
linked_list.insert(5)
linked_list.insert(6)
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:/Users/Abhi/
ts/AI Assistant Coding/Assign 11.4.py"

1

3
3
3
3 6
3
3 7
3 7 9
```

Explanation:

This task implements a linked list supporting insertion, deletion, and traversal while handling edge cases such as empty lists and deletion at the head. The output shows operations executing correctly for multiple scenarios, confirming pointer updates are handled properly (pages 7–8). AI assistance helped by adding comments explaining node links and highlighting tricky pointer transitions. It also suggested testing edge cases like single-node deletion and missing values. This shows AI's usefulness in improving reliability and developer understanding of pointer-based structures.

Task 4: Binary Search Tree for Fast Record Lookup

Task Description:

- **Implement a Binary Search Tree (BST) with:**
  - **insert(value)**
  - **search(value)**
  - **inorder_traversal()**
- **Provide AI with a partially written Node and BST class.**
- **Ask AI to:**
  - **Complete missing methods**
  - **Add meaningful docstrings**
  - **Explain how BST improves search efficiency over linear search**

Prompt:
Implement a binary search tree class with the following methods: insert, delete, and search other methods with meaningful docstrings.

Code:

Assign 11.4.py > ...

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
class BinarySearchTree:
    def __init__(self):
        self.root = None
    def insert(self, data):
        """
        Insert a node with the given data into the binary search tree.
        """
        if self.root is None:
            self.root = Node(data)
        else:
            self._insert_recursive(self.root, data)
    def _insert_recursive(self, node, data):
        if data < node.data:
            if node.left is None:
                node.left = Node(data)
            else:
                self._insert_recursive(node.left, data)
        else:
            if node.right is None:
                node.right = Node(data)
            else:
                self._insert_recursive(node.right, data)
    def delete(self, data):
        """
        Delete a node with the given data from the binary search tree.
        """
        self.root = self._delete_recursive(self.root, data)
    def _delete_recursive(self, node, data):
        if node is None:
            return node
        if data < node.data:
            node.left = self._delete_recursive(node.left, data)
        elif data > node.data:
            node.right = self._delete_recursive(node.right, data)
```

```python
class BinarySearchTree:
    def _min_value_node(self, node):
        current = node
        while current.left is not None:
            current = current.left
        return current
    def search(self, data):
        """
        Search for a node with the given data in the binary search tree.
        """
        return self._search_recursive(self.root, data)
    def _search_recursive(self, node, data):
        if node is None or node.data == data:
            return node
        if data < node.data:
            return self._search_recursive(node.left, data)
        return self._search_recursive(node.right, data)
    def print_inorder(self):
        """
        Print the nodes of the binary search tree in inorder traversal.
        """
        self._print_inorder_recursive(self.root)
    def _print_inorder_recursive(self, node):
        if node is not None:
            self._print_inorder_recursive(node.left)
            print(node.data, end=' ')
            self._print_inorder_recursive(node.right)
# Test cases:
# 1. Insert nodes into the binary search tree
bst = BinarySearchTree()
bst.insert(5)
bst.insert(3)
bst.insert(7)
bst.insert(2)
bst.insert(4)
bst.insert(6)
bst.insert(8)
# 2. Print the binary search tree in inorder traversal
print("Inorder traversal:")
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi\A
Assign 11.4.py"
Inorder traversal:
2 3 4 5 6 7 8
```

Explanation:

This task builds a Binary Search Tree with insert, search, and traversal functions to improve lookup efficiency. The output shows sorted inorder traversal, proving correct BST structure and functioning search logic (page 11). AI explains that BST search runs in **O(log n)** in the best case when the tree is balanced, but **O(n)** in the worst case when it becomes skewed like a linked list. This comparison highlights how performance depends on data distribution and tree balance. Such analysis helps developers understand when BSTs outperform linear search.

Task 5: Graph Traversal for Social Network Connections

Task Description:

- **Represent the network using a graph (adjacency list).**
- **Use AI to implement:**
  - **Breadth-First Search (BFS) to find nearby connections**
  - **Depth-First Search (DFS) to explore deep connection paths**
- **Ask AI to:**
  - **Add inline comments explaining traversal steps**
  - **Compare recursive and iterative DFS approaches**
  - **Suggest practical use cases for BFS vs DFS**

Prompt:

Implement a graph class with the following methods: add_vertex, add_edge, bfs, and dfs with inline comments.

Code:

Assign 11.4.py

Assign 11.4.py > Graph > bfs

```python
# Implement a graph class with the following methods: add_vertex, add_edge, bfs, and dfs with inline comments.
class Graph:
    def __init__(self):
        self.graph = {}
    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []
    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)
    def bfs(self, start_vertex):
        visited = set()
        queue = [start_vertex]
        while queue:
            vertex = queue.pop(0)
            if vertex not in visited:
                print(vertex, end=' ')
                visited.add(vertex)
                queue.extend(self.graph[vertex])
    def dfs(self, start_vertex, visited=None):
        if visited is None:
            visited = set()
        if start_vertex not in visited:
            print(start_vertex, end=' ')
            visited.add(start_vertex)
            for neighbor in self.graph[start_vertex]:
                self.dfs(neighbor, visited)
# Test cases:
# 1. Create a graph and add vertices and edges
graph = Graph()
graph.add_vertex('A')
graph.add_vertex('B')
graph.add_vertex('C')
graph.add_edge('A', 'B')
graph.add_edge('A', 'C')
# 2. Perform breadth-first search (BFS) starting from vertex 'A'
print("Breadth-First Search (BFS) starting from vertex 'A':")
graph.bfs('A')
print()
# 3. Perform depth-first search (DFS) starting from vertex 'A'
print("Depth-First Search (DFS) starting from vertex 'A':")
graph.dfs('A')
print()
```

Output:

```
PS C:\Users\Abhi\Documents\AI Assistant Coding> & C:\Users\Abhi\AppData\
Assign 11.4.py"
Breadth-First Search (BFS) starting from vertex 'A':
A B C
Depth-First Search (DFS) starting from vertex 'A':
A B C
```

Explanation:

This task implements BFS and DFS on a graph to explore nearby and deep connections respectively. The output demonstrates correct traversal orders starting from vertex A, confirming both algorithms work as intended (page 13). BFS is useful for shortest-level exploration such as friend suggestions, while DFS is useful for exploring deeper relationship paths. AI-generated comments help clarify traversal steps and differences between recursive and iterative DFS. This shows how AI can improve both algorithm clarity and practical understanding of graph usage.